Passo 1: Fundamentos Básicos (I/O, Tipos e Operadores)

Foco: `readln()`, conversão de tipos (`.toInt()`, `.toDouble()`) e String Templates (`$variavel`).

1. **Boas-Vindas e Tipos:** Escreva um programa que leia o nome e o salário de um funcionário e imprima uma mensagem usando *String Templates*.

- *Ex:* `O funcionário $nome tem um salário de R$$salario.`

2. **Calculadora Simples:** Leia dois números reais (`Double`) e mostre a soma, subtração, multiplicação e a média entre eles.
3. **Conversor de Medidas:** Leia uma distância em metros e mostre os valores em Quilômetros e Centímetros.
4. **Descontos e Reajustes:** Crie um programa que leia o preço de um produto e o salário de um funcionário. Mostre o preço do produto com 5% de desconto e o salário com 15% de aumento.
5. **Cálculo de Viagem:** Calcule o preço total de um aluguel de carro, perguntando a quantidade de Km percorridos e os dias alugados. (R$90/dia e R$0,20/Km).

Passo 2: Estruturas de Decisão (`if/else` e `when`)

Foco: Condicionais booleanas, `if` como expressão e a poderosa estrutura `when` do Kotlin.

6. **Par ou Ímpar:** Leia um número e diga se é par ou ímpar. *Dica: Tente usar o `if` como expressão: `val resultado = if (num % 2 == 0) "Par" else "Ímpar"`*.
7. **Radar de Velocidade:** Pergunte a velocidade de um carro. Se passar de 80Km/h, mostre que foi multado e o valor (R$5 por cada Km acima do limite).
8. **Alistamento Militar:** Leia o ano de nascimento de uma pessoa. Mostre a sua idade e se ela: ainda vai se alistar (quantos anos faltam), se é o ano de se alistar, ou se já passou do tempo.
9. **Calculadora de IMC (Uso do `when`):** Leia o peso e a altura, calcule o IMC (`peso / altura²`) e use a estrutura `when` para classificar:

- Menor que 18.5: Abaixo do peso
- 18.5 a 24.9: Peso ideal
- 25.0 a 29.9: Sobrepeso
- 30 ou mais: Obesidade

10. **Aprovação de Empréstimo:** Pergunte o valor da casa, o salário e em quantos anos a pessoa vai pagar. A prestação mensal não pode exceder 30% do salário. Informe se foi aprovado ou negado.
11. **[DESAFIO] JoKenPo:** Crie um jogo de Pedra-Papel-Tesoura contra o computador. *Dica: Use `(1..3).random()` para a escolha da máquina.*

Passo 3: Laços de Repetição (`for`, `while` e `Ranges`)

Foco: Operadores de intervalo (`..`, `downTo`, `step`), `while(true)` e interrupções (`break`).

12. **Contagens Dinâmicas (`for`):** Peça ao usuário um valor inicial, um final e o incremento. Mostre a contagem na tela.

- *Dica em Kotlin:* Use `for (i in inicio..fim step incremento)` . Se o final for menor que o início, use `downTo` .

13. **Estatísticas de Grupo (`while`):** Crie um programa que leia a idade e o peso de várias pessoas. O programa deve parar quando o usuário digitar `0` para a idade (uso de *Flag* com `break`). No final, mostre:

- Quantas pessoas foram cadastradas.
- A média de idade do grupo.
- Quantas pessoas pesam mais de 90Kg.

14. **Soma com Regras:** Leia 10 números usando um `for` . No final, mostre o somatório geral, e informe quantos eram pares e quantos eram ímpares.
15. **[DESAFIO] Jogo de Adivinhação:** O computador vai sortear um número entre 1 e 10. O jogador tem 4 tentativas para acertar. A cada erro, o programa deve dizer se o número secreto é maior ou menor que o palpite.

Passo 4: Coleções (`List` , `MutableList` e `Array`)

Foco: Criação de listas, adição de elementos e uso de funções funcionais nativas do Kotlin (`max` , `min` , `filter`).

16. **Preenchimento Automático:** Crie uma `MutableList<Int>` , sorteie 10 números aleatórios (entre 1 e 100) e adicione à lista. Mostre a lista na tela.
17. **Filtros em Listas:** Peça para o usuário digitar 7 números inteiros e guarde-os numa lista. Depois, mostre:

- Todos os valores inseridos.
- Apenas os números pares. *Dica: Use `lista.filter { it % 2 == 0 }*` .
- O maior e o menor valor digitado. *Dica: Use `lista.maxOrNull()` e `lista.minOrNull()*` .

18. **Sequência de Fibonacci em Lista:** Preencha uma lista com os 15 primeiros elementos da sequência de Fibonacci (1, 1, 2, 3, 5, 8...) e depois a exiba.
19. **Busca na Lista:** Preencha uma lista com 20 números aleatórios. Peça ao usuário para digitar uma "chave" (um número). Mostre se a chave existe na lista e, se sim, em quais "índices" (posições) ela aparece.

Passo 5: Funções (`fun`)

Foco: Modularização, parâmetros padrão (default arguments) e retorno de valores.

20. **Função de Texto Visual:** Crie uma função `mostrarTitulo(mensagem: String, borda: String = "=")` que imprima um texto decorado. Repare que a borda deve ser opcional (usando argumentos padrão do Kotlin).

- *Ex:* `mostrarTitulo("Kotlin")` gera `======== \n Kotlin \n ========`.

21. **Função Analisadora:** Crie uma função `maiorNumero(a: Int, b: Int, c: Int): Int` que receba três números e retorne o maior entre eles. Mostre o resultado na tela principal.
22. **Sistema de Notas (Funções Encadeadas):**

- Crie uma função `calcularMedia(n1: Double, n2: Double): Double`.
- Crie outra função `situacaoAluno(media: Double): String` que retorne "Aprovado", "Recuperação" ou "Reprovado".
- No bloco principal (`main`), leia as notas de um aluno, passe para a função de média, e o resultado passe para a função de situação, exibindo o veredito final.

23. **[DESAFIO] Single-Expression Function:** Crie uma função `potencia(base: Double, expoente: Int): Double`. Em Kotlin, se uma função tem apenas uma linha, ela pode ser escrita sem chaves usando o sinal de igualdade. Tente criar essa função usando a classe nativa de matemática: `fun potencia(b: Double, e: Int) = Math.pow(b, e.toDouble())`.